

Ascend C 开发体验升级





该二维码7天内(7月2日前)有效，重新进入将更新

Ascend C 算子编程交流群

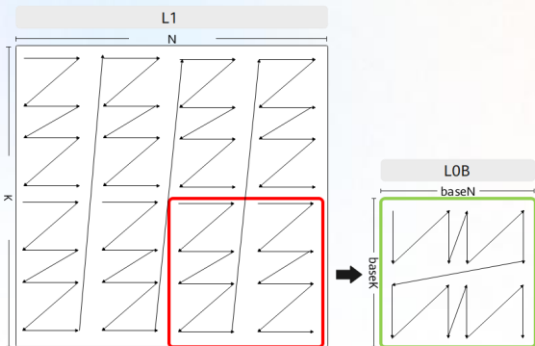
Ascend C交流社区

<https://gitcode.com/cann/asc-devkit>

总览：Ascend C 用户开发体验诉求

Tensor编程重磅推出

- ✓ 传统开发计算**多维内存**索引复杂，边界处理难度高
- ✓ 算法代码与数据布局强耦合，**迁移难度大**



通信编程全新首发

优化**接口易用性**，屏蔽**通路**复杂度，释放通信极致性能

通信驱动单元	优势	劣势
CCU	• 不占用计算核 • 高带宽低时延	• 支持通信域数量有限 • 底层指令复杂度高
AICORE	• 低时延 • 开发方便	• 占用计算核
AICPU	• 不占用计算核 • 大数据量优势	• 头开销大

算子编译极简优化



调试难度高，仿真调试复杂



NPU数据如黑盒，精度难定位



算子编译流程复杂，**交付件多**

手册样例全面重构

入门用户

- ✓ 入门资料简单
- ✓ 调试调优效率低

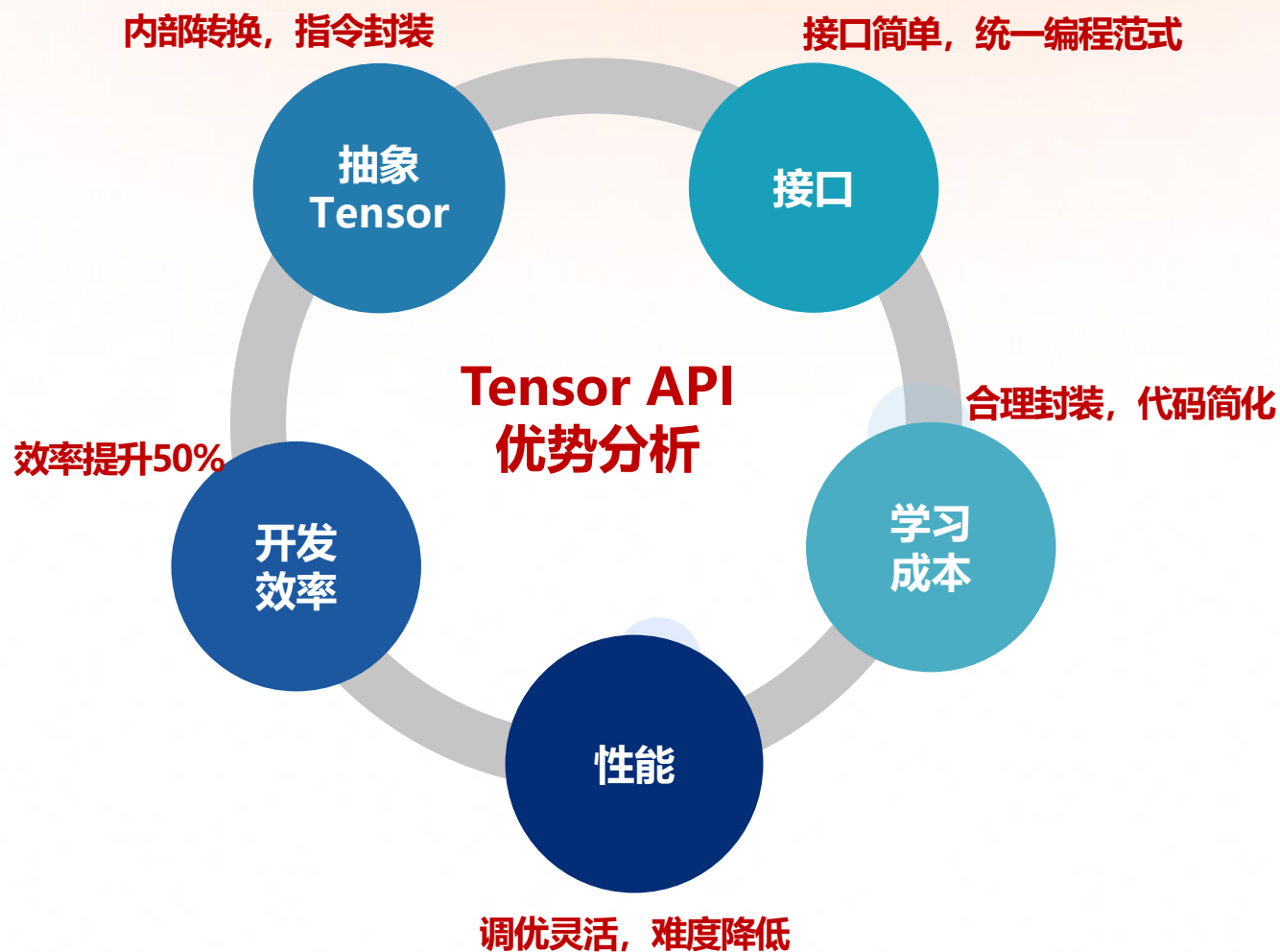
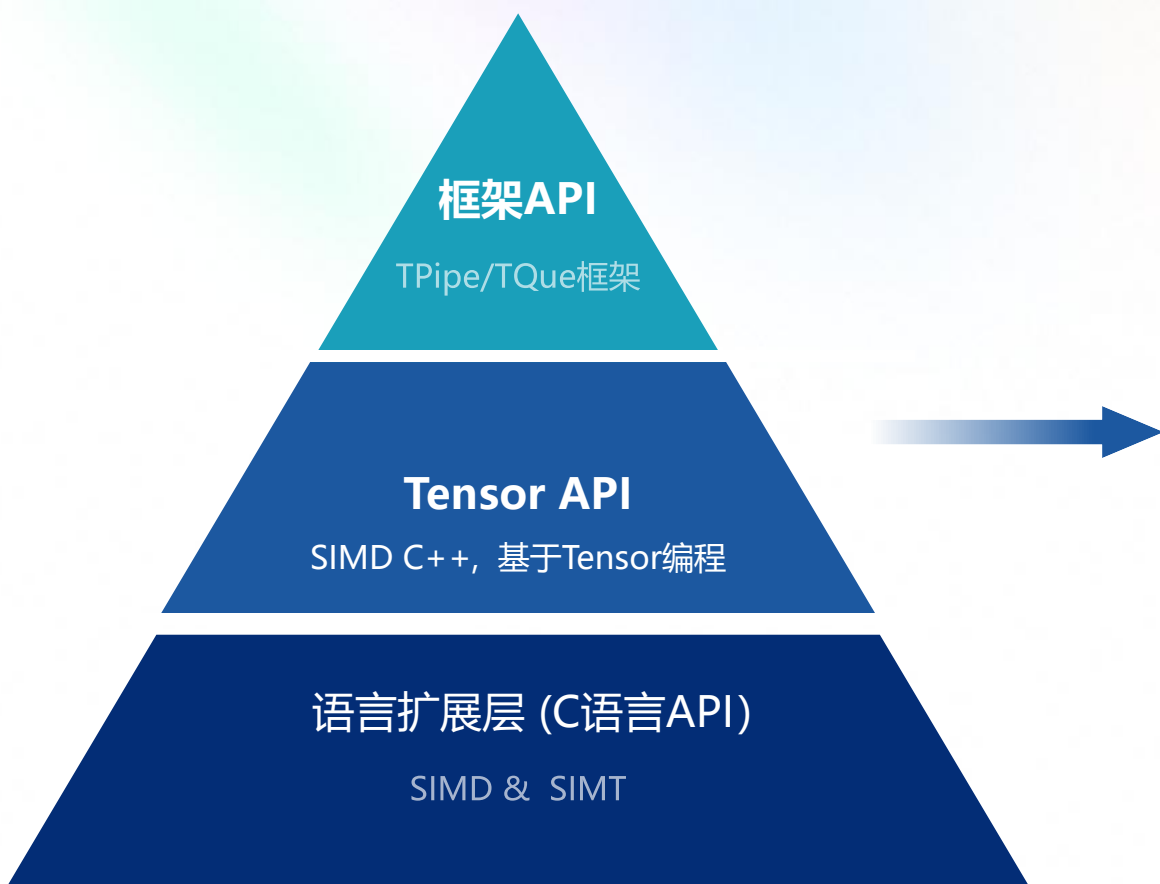
深度用户

- ✓ API资料底层详解
- ✓ 极致性能实际案例

循序渐进

1 Tensor编程：基于Tensor API 全面提升Tensor编程体验

基础API能力拆分为：C++ Tensor编程基础能力(Tensor API)、框架编程



2 Ascend C 通信编程：AIN API

整体规划

完成AIN首版本构建
支持AICORE通信引擎

0630

AIN接口完备度100%
支持CCU通信引擎

0930

完成三种主流通信引擎覆盖
AIN成为HCCL/MC2编程底座

1230

核心定位

两种
主力场景

一套
编程接口

多种
通信引擎

构建AIN通信API，支撑纯通信与通算融合算子编程

通算融合

基础通信

AIN (Ascend-Initiated Networking)

AICPU

CCU

AICORE

通过AIN提供统一通信编程API

竞争力点

易用性

一套API对应多种通信引擎

AICPU, CCU, AICORE多种引擎共用一套API，后续新增引擎也不需修改接口

开发效率

接口形式统一

用户可高效开发通信/通算融合算子

性能

极致压缩头开销

AICORE接口实测头开销小于100ns

3 Ascend C 算子编译：支持混合编译，交付件和仿真模式简化

优化后：Host/Device混合编译，不需要手动区分Host/Device代码，一键式完成异构编译

```
#include "debug/asc_printf.h"
#include "acl/acl.h"

__global__ __vector__ void hello_world()
{
    printf("Hello World!!!\n");
}

int main(int argc, char const* argv[])
{
    aclrtSetDevice(0); // 运行管理资源申请。
    aclrtStream stream = nullptr;
    aclrtCreateStream(&stream);
    hello_world<<<8, nullptr, stream>>>();
    aclrtSynchronizeStream(stream);
    aclrtDestroyStream(stream);
    aclrtResetDevice(0); // 销毁运行资源。
    return 0;
}
```

设备侧编译

Device代码 → fatbin.o

主机侧编译

Host代码 → host.o

链接 → bin/.so/.a

```
void op_hello_world()
{
    auto acl_stream =
c10_npu::getCurrentNPUStram().stream(false);
    hello_world<<<blockDim, nullptr, acl_stream>>>();
    return;
}
TORCH_LIBRARY(ascendc_ops, m)
{
    m.def("hello_world",
    &ascendc_elewise_ops::op_hello_world);
}
```

pytorch调用无需main函数

编译配置

```
cmake_minimum_required(VERSION 3.16)
find_package(ASC REQUIRED)
project(kernel_samples LANGUAGES ASC CXX)

add_executable(add
    add.asc
)

# =====
# NPU 编译选项配置
# 说明:
# - 需根据实际部署的 NPU 硬件架构选择对应的 `npu-arch` 参数。
# =====
target_compile_options(add PRIVATE
    ${CMAKE_COMPILE_LANGUAGE_ASC} --npu-arch=dav-2201>
    # ${CMAKE_COMPILE_LANGUAGE_ASC} --npu-arch=dav-3510>
)
```

交付件简化：编写并验证一个mix算子所需要的交付件变化

```
├─ cmake // CMake配置文件（必需）
│   └─ cpu_lib.cmake
│   └─ npu_lib.cmake
├─ CMakeLists.txt // CMakeLists.txt（必需）
├─ data_utils.h
├─ main.cpp // Host侧调用程序（必需）
├─ matmul_leakyrelu_custom.cpp // mix算子计算逻辑（必需）
├─ matmul_leakyrelu_custom_tiling.cpp // Host侧tiling计算逻辑（必需）
├─ README.md
├─ run.sh // 一键运行脚本（可选）
├─ scripts // 验证脚本
│   └─ gen_data.py
│   └─ verify_result.py
```



```
G matmul_leakyrelu_advanced_api.asc
C data_utils.h
i README.md
M CMakeLists.txt
```

- ✓ <<<>>>调用模式支持
CPU/SIM仿真调试，同一套
代码无缝切换，编译运行
- ✓ 扩展printf能力，支持运行时
kernel侧打印

4 Ascend C手册&样例：了解用户诉求，全面升级开源资料&样例

1

目录结构重新分层

目录结构分层，结合具体业务逻辑，渐进式教学；相关接口整合，方便查找。

2

概述全面升级

概述升级，补充公式和简述，快速选择，信息内聚、减少阅读成本。

3

背景知识充分补充

补充背景知识及关键概念，大幅提升阅读体验。

4

接口全量约束补齐

明确接口能力边界，减少试错成本。

5

关键特性图文结合

关键特性结合图文、公式、伪代码、性能分析，更易于理解。

6

深入底层原理细节

详细展开接口底层原理，方便用户理解指令精度细节及优化过程。

7

理论性能、流水类型补充

提供全量接口理论性能数据，方便开发者分析性能瓶颈。流水类型按接口查表，减少分析时间。

8

结合样例展开说明

关键特性结合样例展开说明，从阅读到实操，开箱即用。

9

接口边界值补充

补充边界值说明，对齐IEEE标准。

帮助开发者更快地查找接口、更好地理解功能、开发更高效的算子

4.1 Ascend C开发手册：补充理论分析、边界值说明

Cube/Vector计算指令理论性能补充、配合Profiling数据，消除性能瓶颈

接口	src数据类型	dst数据类型	理论并行度 (element/cycle)
ReduceDataBlock	half	half	128/7
ReduceDataBlock	float	float	64
ReduceRepeat	half	half	128/7
ReduceRepeat	float	float	64/7
ReducePairElem	half	half	128
ReducePairElem	float	float	64
ReduceMax/ReduceMin/ReduceSum	-	-	软仿指令，不涉及理论并行度

接口	左矩阵A	右矩阵B	cube _m	cube _n	cube _k	k ₀
Mmad	int8_t	int8_t	16	16	32	32
Mmad	half	half	16	16	16	16
Mmad	float	float	16	16	4	8
Mmad	bfloat16_t	bfloat16_t	16	16	16	16
Mmad	int4b_t	int4b_t	16	16	64	64
MmadWithSparse	int8_t	int8_t	16	16	32	32

补充边界值说明，对齐IEEE标准

src/dst的数据类型	src的元素取值	dst的元素取值 (INF/NAN模式)	dst的元素取值 (饱和模式)
half	-65504.0 (MIN)	0.0	0.0
half	65504.0 (MAX)	inf	65504.0
half	0.0	1.0	1.0
half	-0.0	1.0	1.0
half	nan	nan	0.0
half	inf	inf	65504.0
half	-inf	0.0	0.0
float	-3.4028235e+38 (MIN)	0.0	0.0

表 1 模板参数说明

参数名	描述
mode	配置的对饱和模式。 当mode配置为FLOAT时，开启饱和模式，inf输出会被饱和为±MAX，NaN输出会被饱和为0；关闭饱和模式，inf/NaN保持原输出。 当mode配置为CAST时，开启饱和模式，溢出值会被饱和为±MAX；关闭饱和模式，溢出值会按目标数据类型位数截断，保留低位，舍弃高位。 <pre>enum class SaturationMode : uint8_t { FLOAT, // 控制浮点数计算和浮点数精度转换时的饱和模式，浮点数据类型仅支持half、bfloat16_t; CAST // 控制浮点数转整数或整数转整数时的精度转换饱和模式。 };</pre>

表 2 参数说明

参数名	输入/输出	描述
enableSat	输入	是否开启对应饱和模式。true表示开启饱和模式，false表示关闭饱和模式。

4.2 Ascend C开发手册：图文与实践结合，强化理解，开箱即用

Step1：手册理论分析

Bank冲突手册内容节选：

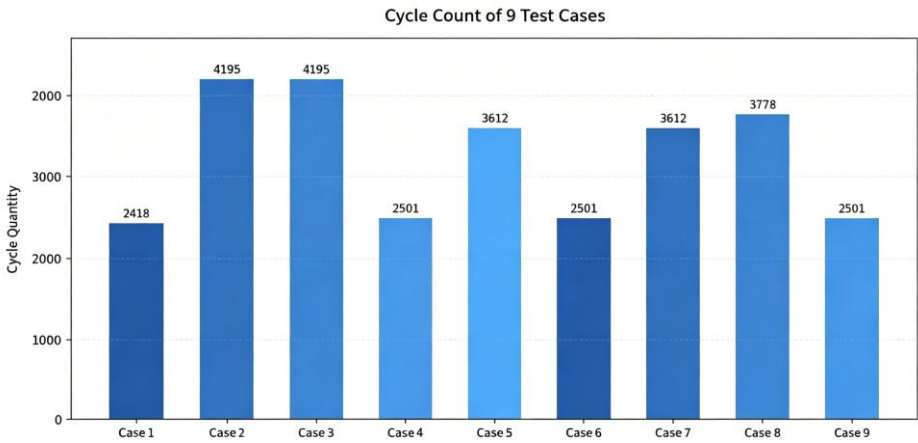
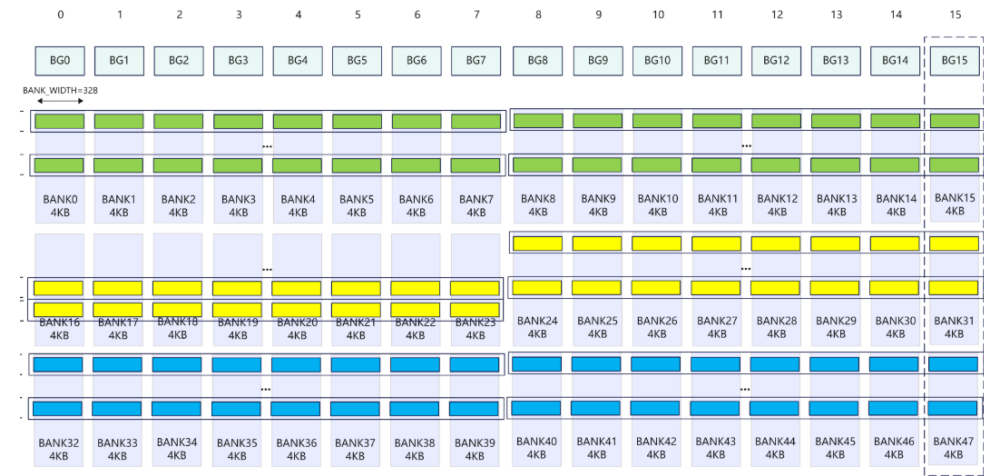
- 读写冲突：读操作和写操作同时尝试访问同一个bank；
 - 写写冲突：多个写操作同时尝试访问同一个bank group；
 - 读读冲突：多个读操作同时尝试访问同一个bank group；
- 性能观测：ResourceConflictRatio（资源冲突占比）；
- 优化建议：地址错开 | 数据重排 | 利用底层优化技术；

Step2：基于样例理解UB Bank冲突过程

计算公式： $dst = src0 * src1$

Case	场景描述	冲突情况	src0地址	src1地址	dst地址
0	src0/src1/dst不同BG不同Bank	无冲突	0x0	0x10100	0x20000
1	src0/src1同Bank	读读冲突	0x0	0x4000	0x10000
2	src0/src1同BG	读读冲突	0x0	0x10000	0x14100
3	src0/dst同Bank	无冲突（底层优化）	0x0	0x10100	0x4000
4	src0/dst不同Bank	读写冲突	0x0	0x10100	0x4020
5	src1/dst同Bank	无冲突（底层优化）	0x0	0x10100	0x10100
6	src1/dst不同Bank	读写冲突	0x0	0x10100	0x10120
7	src0BlkStride=0; src0RepStride=1	读读冲突	0x0	0x10100	0x10120
8	src0BlkStride=0;	无冲突	0x0	0x10100	0x10100

Step3：性能采集、观测结果



4.3 Ascend C开源样例：层级化重构样例体系，助力开发者入门到精通

250+ 独立样例

Atlas A2/A3、Ascend 950PR 兼容

7 大样例模块

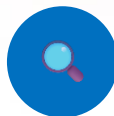
NPU、CPU、仿真 模式支持

SMIT&SIMD / Reg 特性新增

无论是否有NPU计算卡，都可以体验！

目录	样例类别	样例数量	内容定位
00_introduction	入门样例	11	快速上手运行，直观理解原理
01_utilities	调试调测	11	精准定位问题，高效调试排查
02_features	框架特性	7	典型特性示例，编译运行实践
03_basic_api	基础API	93	丰富场景覆盖，彻底吃透特性
04_advanced_api	高阶API	95	高阶接口调用，典型场景实践
05_best_practices	优化实践	16	业务实战落地，性能优化调优
06_compatibility	兼容变更	6	兼容变更适配，旧例迁移参考

- ✓ **层级导引**：区分入门与进阶，降低导航成本
- ✓ **规范补强**：精炼注释、图文结合、丰富场景
- ✓ **梯度构建**：构建入门到精深的最佳优化实践
- ✓ **道术贯通**：授人以渔，引导用户理解原理



找得快 按开发目标进入目录，不靠样例名猜用途



学得顺 从入门到高阶逐层展开，降低首次学习负担



用得准 从接口调用升级为场景参考，按问题选样例



迁得稳 兼容性和运行模式统一说明，减少跨目录比

4.4 Ascend C开源样例：LoadData2DMx-Ascend 950PR矩阵搬运

场景丰富，贴合业务

★表2: scenarioNum不同取值、输入shape及L1->L0搬运方式								
scenarioNum	A / B数据类型	A矩阵类型	B矩阵类型	输入shape	isAtranspose	isBtranspose	A矩阵搬运	B矩阵搬运
1	FP4	fp4x2_e1m2_t	fp4x2_e2m1_t	A [40, 70] B [50, 70] scaleA [40, 4] scaleB [50, 4]	false	true	LoadData2DParamsV2 (不转置)	LoadData2DParamsV2 (不转置)
2	FP4	fp4x2_e2m1_t	fp4x2_e1m2_t	A [70, 40] B [70, 50] scaleA [4, 40, 2] scaleB [4, 50, 2]	true	false	LoadData2DParamsV2 (转置)	LoadData2DParamsV2 (转置)
3	FP8	fp8_e4m3fn_t	fp8_e5m2_t	A [40, 70] B [50, 70] scaleA [40, 4] scaleB [50, 4]	false	true	LoadData2DParamsV2 (不转置)	LoadData2DParamsV2 (不转置)
4	FP8	fp8_e5m2_t	fp8_e4m3fn_t	A [70, 40] B [70, 50] scaleA [4, 40, 2] scaleB [4, 50, 2]	true	false	LoadData2DParamsV2 (转置)	LoadData2DParamsV2 (转置)
5	FP4	fp4x2_e2m1_t	fp4x2_e1m2_t	A [70, 40] B [70, 50] scaleA [4, 40, 2] scaleB [4, 50, 2]	true	false	for循环 + LoadData2DParamsV2 (转置)	LoadData2DParamsV2 (转置)
6	FP8	fp8_e5m2_t	fp8_e4m3fn_t	A [70, 40] B [70, 50] scaleA [4, 40, 2] scaleB [4, 50, 2]	true	false	for循环 + LoadData2DParamsV2 (转置)	LoadData2DParamsV2 (转置)

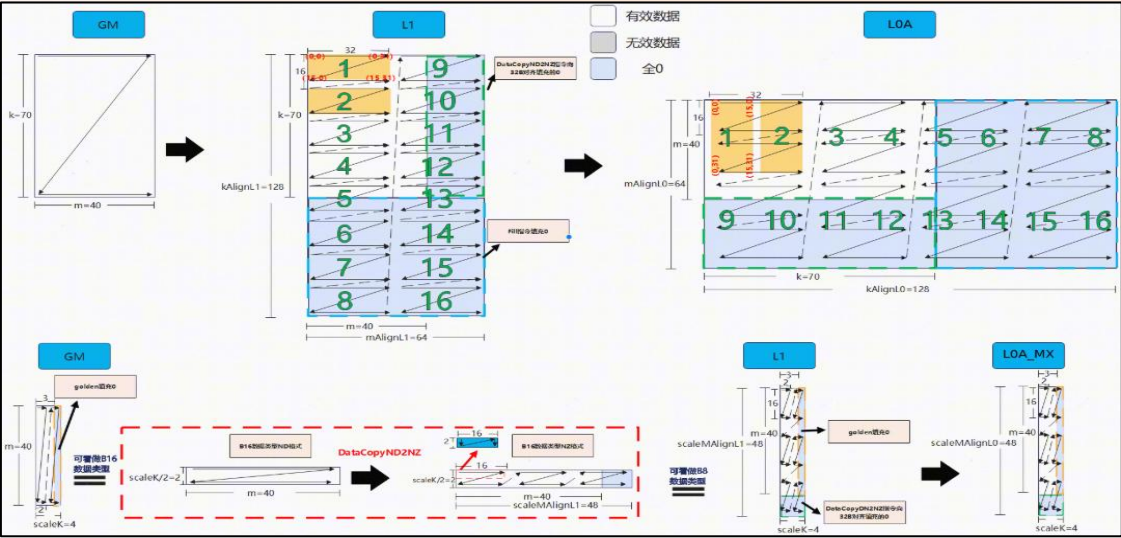
核心配置，注释清晰

DataCopy (Nd2NzParams)

- **dValue**: 取 packedK。FP4输入按B8视图搬运，k方向每2个FP4元素对应1个B8视图元素；FP8输入按原始k配置。
- **dstNzC0Stride**: 单位为32B，取L1上Nz矩阵对齐后的行数，即 mAlignL1。
- **FP4处理**: 根据接口约束，FP4输入在ND -> Nz过程中按B8视图处理，参数配置也按B8视图设置。

```
AscendC::Nd2NzParams nd2nzA1Params;  
nd2nzA1Params.ndNum = 1; // ND矩阵的数目  
nd2nzA1Params.nValue = m; // 源操作ND矩阵的行数  
nd2nzA1Params.dValue = packedK; // 源操作ND矩阵按B8视图配置的列数  
nd2nzA1Params.srcNdMatrixStride = 0; // 源操作数相邻ND矩阵间的起始地址偏移  
nd2nzA1Params.srcDValue = packedK; // 源操作数同一ND矩阵内相邻行的起始地址偏移，按B8视图配置  
nd2nzA1Params.dstNzC0Stride = mAlignL1; // ND -> Nz后，同一行数据切分后各段的起始地址间隔，单位32B  
nd2nzA1Params.dstNzStride = 1; // ND矩阵第x行与第x+1行转Nz后在dst中的偏移  
nd2nzA1Params.dstNzMatrixStride = 0; // 目的Nz矩阵中，相邻Nz矩阵起始地址的偏移，配置为0时，无意义  
AscendC::DataCopy(a1Local, aGM, nd2nzA1Params);
```

图文结合，便于理解



配置齐全，快速上手

使用 CPU调试 或 NPU仿真 模式时，添加 -DCMAKE_ASC_RUN_MODE=cpu 或 -DCMAKE_ASC_RUN_MODE=sim 参数即可。

示例如下：

```
cmake .. -DCMAKE_ASC_RUN_MODE=cpu -DCMAKE_ASC_ARCHITECTURES=dav-3510 -DSCENARIO_NUM=$SCENARIO_NUM;make -j; # CPU调试模式  
cmake .. -DCMAKE_ASC_RUN_MODE=sim -DCMAKE_ASC_ARCHITECTURES=dav-3510 -DSCENARIO_NUM=$SCENARIO_NUM;make -j; # NPU仿真模式
```

注意：切换编译模式前需清理 cmake 缓存，可在 build 目录下执行 rm CMakeCache.txt 后重新 cmake。

编译选项说明

选项	可选值	说明
CMAKE_ASC_RUN_MODE	npu (默认)、cpu、sim	运行模式：NPU运行、CPU调试、NPU仿真
CMAKE_ASC_ARCHITECTURES	dav-3510 (默认)	NPU架构：dav-3510 对应 Ascend 950PR/Ascend 950DT
SCENARIO_NUM	1 (默认)、2、3、4、5、6	场景编号

4.5 Ascend C开源样例：最佳实践，Add的性能“修炼”之路

逐步展示优化进程

图文解释优化原理

Case	优化策略	核数	dataCopyLen	Task Duration(μs)	aiv_vec_time(μs)
0	单核标量 (基准)	1	4096	1239689.1	NA
1	单核向量	1	4096	6909.6	761.65
2	多核均匀切分	48	4096	306.58	15.885
3	增大搬运粒度	48	16384	268.5	12.845
4	双缓冲	48	16384	264.02	12.846
5	L2 Cache bypass	48	16384	187.68	12.846
6	Bank Conflict优化	48	16256	184.52	6.954

理论分析性能上限

表中的“理论vector耗时”表示在当前核数配置下，仅考虑Vector计算本身时的理论执行时间。本样例的性能数据在Atlas A2训练系列产品上运行，该处理器每cycle处理128个half数据，主频为1.85GHz。理论vector耗时的计算公式为

$$T_{\text{theory}} = \frac{M \times N}{128 \times 1.85 \times 10^9 \times \text{核数}}$$

例如，48 核场景下：

$$T_{\text{theory}} = \frac{8192 \times 8192}{128 \times 1.85 \times 10^9 \times 48} = \frac{67108864}{1.13664 \times 10^{13}} \approx 5.904 \times 10^{-6} \text{ s} = 5.904 \text{ } \mu\text{s}$$

可以看到 Case 6 的 aiv_vec_time 为 6.954 μs，已经很接近 48 核场景下的理论耗时。

Case 0-3 未开启双缓冲，数据搬运串行执行，使用读带宽衡量性能。从 Case 4 开始开启双缓冲，此时 mte2 利用率较高，读写行为并行发生，因此读写混合带宽按总读写数据量除以 T_{mte2} 进行估算。其中，读带宽的计算公式为：

$$BW_{\text{read}} = \frac{D_{\text{read}}}{T_{\text{mte2}}}$$



完整保留实现过程

• 编译选项说明

选项	可选值	说明
CMAKE_ASC_RUN_MODE	npv（默认）、cpu、sim	运行模式：NPU 运行、CPU调试、NPU仿真
CMAKE_ASC_ARCHITECTURES	dav-2201（默认）、dav-3510	NPU 架构：dav-2201 对应 Atlas A2 训练系列产品/Atlas A2 推理系列产品和 Atlas 品，dav-3510 对应 Ascend 950PR/Ascend 950DT
SCENARIO_NUM	0（默认）、1、2、3、4、5、6	性能优化case编号

执行结果如下，说明精度对比成功。

error ratio: 0.0000, tolerance:0.0001
test pass!

4.6 Ascend C开源样例：Ascend 950PR场景样例全面支持

MXFP4矩阵运算场景

MxMatmul 在此基础上额外引入两路 scale 输入。A、B 为 MxFP4 低比特数据，scaleA、scaleB 为缩放因子。计算时，矩阵数据与对应 scale 共同参与运算：

$$C = (\text{scaleA} \otimes A) \times (\text{scaleB} \otimes B)$$

在MxMatmul中，K 方向每 32 个元素共享一个 scale。也就是说，scale 的数量远少于矩阵元素数量，但它会沿 K 方向广播到对应的一组 MxFP4 数据上，如图所示：

Diagram illustrating the MxMatmul operation with scale factors. Matrix A (MxK) is multiplied by Scale A (Kx1), and Matrix B (KxN) is multiplied by Scale B (Kx1). The results are then multiplied together to produce Matrix C (MxN). Scale A and Scale B are shared across 32 elements in the K dimension.

RegBase编程场景

IPC对比总结

Case	优化策略	vector耗时(μs)	IPC	IPC相对Case 0
0	单loop计算，并行度128 bytes（基准）	244.299	1.06	1x
1	单loop计算，并行度256 bytes	125.203	1.03	0.97x
2	Loop拆分（Gelu + Element-wise）	97.184	1.33	1.25x
3	Loop拆分 + unroll循环展开	94.339	1.37	1.29x

IPC优化建议：

- IPC越高越好，理论极限接近2.0，大多数情况下达成1.4~1.5则表示取得较优性能

SIMT&SIMD混合编程场景

对比	线程块个数	vf次数变化	Task Duration变化	增幅
SCENARIO_NUM=2→6	32	1→2	4.413 → 4.503	+2.0%
SCENARIO_NUM=3→7	64	1→2	4.602 → 4.869	+5.8%
SCENARIO_NUM=5→8	1024	1→2	13.687 → 14.982	+9.5%

性能数据如下图所示：

Case 0/1 调度开销：线程块数量与vf调用次数

Thread Block Count	1xvf调用 (μs)	2xvf调用 (μs)
1	3.933	
32	4.413	4.503
64	4.602	4.869
128	5.269	
1024	13.687	14.982

Cube/Vector融合场景

Scenario 2: CV融合-UB直通

以Ascend 950PR芯片数据为例：

- Scenario 2融合后总耗时2573.584μs，相比Scenario 1的2606.91μs进一步减少约33μs
- Scenario 2中aiv_mte2_time仅为0.005μs，相比Scenario 1的57.145μs几乎降为零

下图展示了Scenario 2（CV融合-UB直通）场景下AIC和AIV的流水并行执行过程。AIC侧完成Matmul计算后通过Fixpipe将结果直接写入UB（不经过GM中转），AIV侧直接在UB上进行Gelu计算，省去了GM→UB的MTE2搬运开销：

THANKS



该二维码7天内(7月2日前)有效, 重新进入将更新

Ascend C 算子编程交流群